La serialización de objetos en Java

Java ha añadido una interesante faceta al lenguaje denominada **serialización de objetos** que permite convertir cualquier objeto cuya clase implemente **el interface Serializable** en una secuencia de bytes que pueden ser posteriormente leídos para restaurar el objeto original.

Esta característica se mantiene incluso a través de la red, por lo que podemos crear un objeto en un ordenador que corra bajo Windows, serializarlo y enviarlo a través de la red a una estación de trabajo que corra bajo UNIX donde será correctamente reconstruido. No tenemos que preocuparnos, en absoluto, de las diferentes representaciones de datos en los distintos ordenadores.

La **serialización** de un objeto consiste en generar una secuencia de bytes lista para su almacenamiento o transmisión. Después, mediante la **deserialización**, el estado original del objeto se puede reconstruir. Para que un objeto sea serializable, ha de implementar la interfaz **java.io.Serializable** (que lo único que hace es marcar el objeto como serializable, sin que tengamos que implementar ningún método).

La serialización es una característica añadida al lenguaje Java para dar soporte a

- La invocación remota de objetos (RMI)
- La persistencia

La invocación remota de objetos permite a los objetos que viven en otros ordenadores comportarse como si vivieran en la propia máquina. La serialización es necesaria para transportar los argumentos y los valores de retorno.

La persistencia, es una característica importante de los **JavaBeans**. El estado de un componente es configurado durante el diseño. La serialización nos permite guardar el estado de un componente en disco, abandonar el Entorno Integrado de Desarrollo (IDE) y restaurar el estado de dicho componente cuando se vuelve a correr el IDE.

El interface Serializable

Un objeto se puede serializar si implementa el interface Serializable. Este interface no declara ningún método, se trata de un interface vacío.

```
import java.io.*;

public interface Serializable{ }
```

La interfaz Serializable no contiene ningún método para que se implemente

```
import java.io.Serializable;
import java.util.Date;

public class Contacto implements Serializable
{
private String nombre;
private String telefono;
private String email;
private String direccion;
private Date nacimiento;
private int grupo;
private double deuda;
...
}
```

Ejemplo de una clase que implementa la interfaz Serializable

- Para que un objeto sea serializable, todas sus variables de instancia han de ser serializables.
- Todos los tipos primitivos en Java son serializables por defecto (igual que los arrays y otros muchos tipos estándar).

Un ejemplo de una clase serializable se muestra a continuación. En este ejemplo la serialización consiste en abrir un fichero "clientes.dat" y escribir una serie los objetos definidos como serializables, en este caso definidos en la clase "Contacto".

Serialización

```
import java.io.*;

public class ContactoOutput  {

        private FileOutputStream file;
        private ObjectOutputStream output;

        // Abrir el fichero
        public void abrir() throws IOException {
        file = new FileOutputStream( "clientes.ser" );
        output = new ObjectOutputStream(file);
        }

        // Cerrar el fichero
        public void cerrar() throws IOException {
          if (output!=null)
             output.close();
        }

        // Escribir en el fichero
        public void escribir (Contacto contacto) throws IOException {
           if (output!=null)
           output.writeObject(contacto);
        }
}
```

Por otro lado, el proceso de deserialización consistirá en la lectura de los objetos serializados del fichero “contactos.dat” y mostrarlos correctamente. El fichero con los objetos serializados contactos.dat almacena los datos en un formato propio de Java, por lo que no se puede leer fácilmente con un simple editor de texto (ni editar).

Deserialización

```
import java.io.*;

public class ContactoInput {
        private FileInputStream file;
        private ObjectInputStream input;
        public void abrir()  throws IOException {
           file = new FileInputStream( "contactos.dat" );
           input = new ObjectInputStream (file);
        }

        public void cerrar() throws IOException {
           if (input!=null )
              input.close();
        }

        public Contacto leer () throws IOException, ClassNotFoundException {
           Contacto contacto = null;
           if (input!=null) {
              try {
                 contacto = (Contacto) input.readObject();
              } catch (EOFException eof)
                 {// Fin del fichero }
           }
           return contacto;
        }
}
```

El modificador transient

Cuando un dato de una clase contiene información sensible, hay disponibles varias técnicas para protegerla. Incluso cuando dicha información es privada (el miembro dato tiene el modificador **private**) una vez que se ha enviado al flujo de salida alguien puede leerla en el archivo en disco o interceptarla en la red.

El modo más simple de proteger la información sensible, como una contraseña (password) es la de poner el modificador **transient** delante del miembro dato que la guarda.

En el siguiente ejemplo, la clase **Cliente** tiene dos miembros; dato, el nombre del cliente y la contraseña.

Redefine la función **toString()** método de la clase base **Object**. Esta función devolverá el nombre del cliente y la contraseña. En el caso de que la variable password guarde el valor **null** se imprimirá el texto **(no disponible)**.

En el cuadro que sigue se muestra el código que define la clase **Cliente**.

```
public class Cliente implements java.io.Serializable{

  private String nombre;
  private transient String passWord;

  public Cliente(String nombre, String pw) {
    this.nombre=nombre;
    passWord=pw;
  }

  public String toString(){
    String texto=(passWord==null) ? "(no disponible)" : passWord;
    texto+=nombre;
    return texto;
  }
}
```

Por otro lado el código gestionador de los objetos serializables se implementa en la clase **Serializable**.

```
import java.io.*;

public class Serializacion{

   public static void main(String[] args) {
      Cliente cliente=new Cliente("Jesus", "abc");
      try{
         ObjectOutputStream salida=new ObjectOutputStream
                                        (new FileOutputStream("cliente.obj"));
         salida.writeObject("Datos del cliente\n");
         salida.writeObject(cliente);
         salida.close();

         ObjectInputStream entrada=new ObjectInputStream
                                        (new FileInputStream("cliente.obj"));
         String str=(String)entrada.readObject();
         Cliente obj1=(Cliente)entrada.readObject();
         System.out.println("----------------------------");
         System.out.println(str+" " +obj1);
         System.out.println("----------------------------");
         entrada.close();
        //se puede fundir en una catch Exception
      }catch (IOException ex)
        { System.out.println(ex); }
       catch (ClassNotFoundException ex)
        { System.out.println(ex); }

     try  {
        //espera la pulsación de una tecla y luego RETORNO
         System.in.read();
      }catch (Exception e) {  }
   }
}
```

El programa en sí crea un archivo llamado cliente.obj y escribe el dato del objeto Cliente creado. Posteriormente lee dicho archivo y muestra los datos contenidos.

La salida del programa es

Lo que nos indica que la información sensible guardada en la variable password que tiene por modificador **transient** no ha sido guardada en el archivo. En la reconstrucción del objeto *obj1* con la información guardada en el archivo el miembro dato password toma el valor **null**.

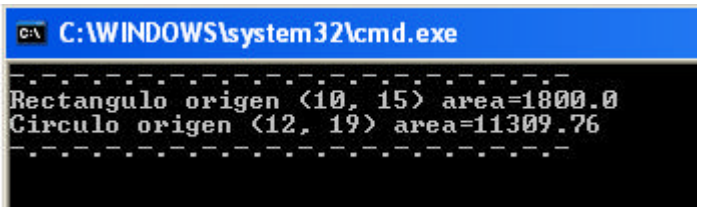
La herencia en objetos serializables

Para serializar objetos de una jerarquía solamente la clase base tiene que implementar el interface Serializable. En el siguiente ejemplo se muestra una herencia de la clase Figura que implementa la clase Serializable. De esta clase heredan la clase **Circulo** y **Rectangulo**, ninguna de ellas implementa la clase Serializable.

```
public abstract class Figura implements java.io.Serializable{
   protected int x;
   protected int y;
   public Figura(int x, int y) {
      this.x=x;
      this.y=y;
   }
   public abstract double area();
}

class Circulo extends Figura{
   protected double radio;
   private static final double PI=3.1416;
   public Circulo(int x, int y, double radio){
      super(x,y);
      this.radio=radio;
   }
   public double area(){
      return PI*radio*radio;
   }
}

class Rectangulo extends Figura{
   protected double ancho, alto;
   public Rectangulo(int x, int y, double ancho, double alto){
      super(x,y);
      this.ancho=ancho;
      this.alto=alto;
   }
   public double area(){
      return ancho*alto;
   }
}
```

```
public class HerenciaSerializable {

   public static void main(String[] args) {
      Figura fig1=new Rectangulo(10,15, 30, 60);
      Figura fig2=new Circulo(12,19, 60);

      try  {
         ObjectOutputStream salida=new ObjectOutputStream
                                          (new FileOutputStream("figura.obj"));
         salida.writeObject("guardar un objeto de una clase derivada\n");
         salida.writeObject(fig1);
         salida.writeObject(fig2);
         salida.close();

         ObjectInputStream entrada=new ObjectInputStream
                                           (new FileInputStream("figura.obj"));
         String str=(String)entrada.readObject();
         Figura obj1=(Figura)entrada.readObject();
         Figura obj2=(Figura)entrada.readObject();
         System.out.println("-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-");
         System.out.println(obj1.getClass().getName()+
                        " origen ("+obj1.x+", "+obj1.y+")"+
                        " area="+obj1.area());
         System.out.println(obj2.getClass().getName()+
                         " origen ("+obj2.x+", "+obj2.y+")"+
                         " area="+obj2.area());
         System.out.println("-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-");
         entrada.close();
         //se puede fundir en una catch Exception
      }catch (IOException ex)
         { System.out.println(ex); }
       catch (ClassNotFoundException ex)
         { System.out.println(ex); }

      try  {
         //espera la pulsación de una tecla y luego RETORNO
         System.in.read();
      }catch (Exception e) {  }
   }
}
```

Vamos a serializar dos objetos uno de la clase **Rectangulo** y otro de la clase **Circulo**, y a continuación reconstruiremos dichos objetos. Una vez que dispongamos de los objetos llamaremos a las funciones **area** para calcular el área de cada una de las figuras. La ejecución de este ejemplo produce la siguiente salida:



Los dos objetos creados (obj1 y obj2) heredan la serialización del la clase Figura por lo que la reconstrucción de estos objetos se realiza sin ningún problema.

Serialización personalizada

El proceso de serialización proporcionado por el lenguaje Java es suficiente para la mayor parte de las clases, ahora bien, se puede personalizar para aquellos casos específicos.

Para personalizar la serialización, es necesario definir dos métodos **writeObject** y **readObject**. El primero, controla que información es enviada al flujo de salida. La segunda, lee la información escrita por writeObject .

La definición de **writeObject** ha de ser la siguiente:

```
private void writeObject (ObjectOutputStream s) throws IOException{
        s.defaultWriteObject();
        //...código para escribir datos
}
```

La función **readObject** ha de leer todo lo que se ha escrito con **writeObject** en el mismo orden en el que se ha escrito. Además, puede realizar otras tareas necesarias para actualizar el estado del objeto.

```
private void readObject (ObjectInputStream s) throws IOException{
        s.defaultReadObject();
        //...código para leer datos
        //...
        //actualización del estado del objeto, si es necesario
}
```

Para un control explícito del proceso de serialización la clase ha de implementar el interface **Externalizable**. La clase es responsable de escribir y de leer su contenido, y ha de estar coordinada con sus clases base para hacer esto.

La definición del interface **Externalizable** es la siguiente:

```
package java.io;

public interface Externalizable extends Serializable{
  public void writeExternal(ObjectOutput out) throws IOException;
  public void readExternal(ObjectOutput in) throws IOException,
                                                java.lang.ClassNotFoundException;
}
```